COS30019 Assignment 2 Part A

Route Finding with Six Search Methods

Implementation testing and comparative analysis

Submission deadline 25 October 2026 at 11 59 pm

Team ID [Enter team ID]

Team name [Enter team name]

Tutorial group [Enter tutorial group]

Group members and contributions

Table A Team details to be completed by the group

Full name	Student ID	Actual contribution
[Member 1]	[ID]	[Confirm contribution]
[Member 2]	[ID]	[Confirm contribution]
[Member 3]	[ID]	[Confirm contribution]
[Member 4 if applicable]	[ID]	[Confirm contribution]

Member signatures _____

_____ Date _____

Each member must confirm their own contribution and provide their own signature. Remove the fourth row if the group has three members.

The implementation and measured results are documented below. The group must review the report, complete the personal details, and confirm the research component before submission.

Contents

1 Program instructions 3

2 Problem and implementation 4

3 Uninformed search methods 5

4 Informed search methods 6

5 Testing and verification 7

6 Results and interpretation 8

7 Research and limitations 9

8 Conclusion and acknowledgements 10

Report scope

This report explains the six search methods implemented for the route-finding assignment and compares their behaviour on fifteen directed graph problems. The emphasis is on the distinction between lowest cost and fewest moves, correct handling of cycles, and the interpretation of measured search effort.

All ninety method and test-case combinations were checked. Fourteen cases have reachable destinations and one deliberately has none. Returning no route in that case is the correct outcome. The measured results support recommendations for these implementations, rather than universal performance claims.

Supporting project files

The full measurements are stored in results/benchmark.json and results/comparison.md. The benchmark_report.py script reproduces the validation and timing measurements. These supporting files contain the complete ninety-row evidence; the main report selects examples that explain the most important differences.

Page numbers refer to this version of the report. Update the contents after changes to the text, figures or team details.

1 Program instructions

The command-line program reads a graph file and runs one selected method. It returns a path from the origin to any one destination. The program uses Python and standard-library data structures; it does not call a library implementation of a search algorithm. The project was tested on Windows 11 using Python 3.13.14 [2].

Running a search

Open a terminal in the project directory containing search.py. Use the following command, replacing the filename and method as required:

python search.py tests/cases/test01_spec_example.txt AS

The accepted method codes are DFS, BFS, GBFS, AS, CUS1 and CUS2. The Windows search.bat launcher forwards the same two arguments. A filename containing spaces must be enclosed in quotation marks. No explanatory flag is required or supported.

Reading the required output

A successful run produces the three-line format required by the brief [1]:

filename method
goal number_of_nodes
path

For the supplied example, AS reaches destination 5 with nine search nodes created. The final path is 2 3 5. The first number on the second line is a destination label, not a cost. The second number measures created search nodes, not moves in the final route.

The path includes the origin and destination. A route containing three labels has two moves. Its travel cost is the sum of the two directed edge costs. A search-node count can exceed the number of graph locations because different routes may generate records for the same location.

Testing and collecting evidence

Run python -m unittest discover tests to execute the regression suite. Run python run_tests.py to print results for all methods and cases. Run python benchmark_report.py to verify the results against independent reference calculations and regenerate the report evidence [2].

When no destination is reachable, the program reports that no goal is reachable and includes the created-node count. Incorrect method codes, invalid inputs and missing files produce error messages. A GUI file is present in the repository, but GUI behaviour has not been validated in this work and is not part of the reported evidence.

2 Problem and implementation

The input describes a directed weighted graph. Each node has an integer label and two-dimensional coordinates. Each edge specifies a permitted direction and a nonnegative travel cost. The origin identifies the starting node, and the destination set contains one or more acceptable endpoints. The required goal is to reach any one destination, not to visit all destinations [1].

Graph states and search nodes

A graph state is a location such as node 3. A search node is a record of one route to that location: it contains the state, parent record, accumulated path cost and depth. Following parent references reconstructs the final path. Keeping these concepts separate explains why multiple search nodes can refer to a single graph state.

RouteProblem stores coordinates, sorted outgoing edges and the destination set. The parser reads the Nodes, Edges, Origin and Destinations sections, including decimal coordinates and costs. Successors are generated in ascending label order. The search functions return a SearchResult containing either the final search node or None, together with the created-node count [2].

Counting and cycle handling

Every constructed Node counts, including the root and children immediately rejected as duplicates or cycles. BFS was corrected to increment the counter before rejecting an already reached state. This changes the supplied-example BFS count from four to five without changing its route.

BFS keeps a global reached set, avoiding repeated exploration of a state. The other five methods reject a child only when its state already occurs among its ancestors. They can explore the same state through different branches. On a finite graph, restricting routes to simple paths prevents cycles from causing an endless search, but the number of alternative paths can still be large.

Tie breaking and goal timing

DFS pushes siblings in descending order so that the smallest label is popped first. BFS inserts ascending siblings into a first-in first-out queue. Priority methods insert ascending siblings and use insertion order when their numerical priorities tie. This implements ascending sibling order and chronological order across branches; it does not impose a global node-label ordering across every tied branch.

BFS tests goals on generation. DFS and priority methods test goals on removal from the frontier. Cost-sensitive searches must not stop merely because an expensive goal has been generated while a cheaper route remains waiting. These implementation differences matter when comparing counts and timing.

3 Uninformed search methods

Uninformed methods do not use coordinates to estimate how far a destination is. This project includes DFS, BFS and uniform-cost search. They share the same input model and result structure, but their frontier rules lead to different routes [2].

Depth first search

DFS uses a stack. It removes the most recently added node and follows the smallest available child deeper into the graph. If that branch ends without a destination, another waiting branch is explored. Parent-path checking rejects loops such as returning from node 3 to an ancestor node 1.

The supplied example produces 2 -> 1 -> 3 -> 5 with cost 15, three moves and nine created nodes. This route is valid, although other methods find routes costing 10. DFS stops at the first goal it reaches and does not compare that route with every alternative. It therefore offers no guarantee of minimum cost or fewest moves.

Breadth first search

BFS uses a queue and explores all shallower routes before deeper routes. Its global reached set is safe for the fewest-move objective because the first discovery of a state is at minimum depth. The method ignores differences between edge costs. In the example, BFS returns 2 -> 1 -> 4, costing 10 in two moves, with five created nodes.

The equal-priority convention applies within this queue behaviour. Smaller siblings are generated first, while children from an earlier branch precede children generated later. This is not a global sorting of every node at the same depth by its label.

CUS1 as uniform cost search

CUS1 uses a priority queue ordered by accumulated cost g(n). It considers the cheapest waiting partial route first, without consulting geometric distance. With finite nonnegative edge costs and path cycle checking, the first goal removed has minimum travel cost. A path with more moves can still be preferred if its total cost is lower.

In the example, CUS1 returns 2 -> 1 -> 4 with cost 10 and fifteen created nodes. The different route returned by AS has the same cost, so both can be correct. Uniform-cost search is useful as a cost-based baseline because it makes no assumption that physical distance resembles edge cost.

Relationship to established algorithms

The AIMA algorithm reference describes queue-based breadth-first search, priority-based uniform-cost search and stack-based depth-limited search [3]. This implementation uses explicit parent records and the cycle and counting rules stated in Section 2; its measured behaviour should be interpreted using those exact choices.

4 Informed search methods

Informed methods use node coordinates to guide the frontier. Coordinates are geometric information, whereas edge costs may represent a different quantity. A distance estimate must therefore be interpreted carefully before it is added to cost or move count [2].

Greedy best first search

GBFS ranks a node by its straight-line distance to the nearest destination. It ignores the cost already paid to reach that node. This can direct the search towards a plausible endpoint quickly, but a visually close node may connect to the destination through an expensive edge. GBFS is not guaranteed to minimize either cost or moves.

A star search for minimum cost

AS ranks nodes by f(n) = g(n) + h(n). Here g(n) is accumulated edge cost. Let s be the minimum of 1 and cost(u,v) / distance(u,v) over positive-length edges. The implementation sets h(n) to s times straight-line distance to the nearest destination.

For every edge, its cost is at least s times its geometric length. Along any route, summing these inequalities and applying the triangle inequality shows that the scaled direct distance is no larger than the remaining route cost. Taking the nearest destination preserves that lower-bound property. The estimate is therefore safe for the minimum-cost search under finite nonnegative costs.

If a positive-length edge costs zero, s becomes zero and the search behaves like uniform-cost search. Coincident endpoints require no division. Scaling is calculated for each problem, so an unusually cheap edge anywhere in the graph may weaken the estimate even when that edge is not on the solution route.

CUS2 as A star for minimum moves

CUS2 changes the objective to the number of moves. Its priority is depth(n) plus straight-line distance to the nearest goal divided by L, the longest geometric edge length in the graph. One move covers at most L units of straight-line distance; therefore k moves can cover at most kL. The quotient is a lower bound on the moves remaining.

If no edge has positive length, the estimate is zero. The method still records travel cost for reporting, but cost does not affect its priority. Goals are tested on removal and ancestor cycles are rejected. This method can deliberately choose a more expensive route when it requires fewer moves.

Both informed A* variants use the shared priority frontier and retain the same insertion-order ties. Their correctness arguments depend on the relevant lower bound: cost for AS and moves for CUS2. These are separate objectives and should not be combined into a single ranking of route quality.

5 Testing and verification

Fifteen input files cover different graph structures and objectives. All six methods were run on every input, giving ninety checked combinations. The regression suite contains thirty-eight test methods; several methods loop over the full fixture set, so this count is distinct from the number of graph cases [2].

Table 1 Coverage of the fifteen supplied cases

Case	Scenario	Purpose
01	Supplied example	Direction and known routes
02	Origin is a goal	Zero-move solution
03	Unreachable goal	Correct failure
04	One-way edges	Respect direction
05	Equal-priority branches	Sibling ordering
06	Cheap long route	Cost versus moves
07	Multiple destinations	Reach any goal
08	Cycle trap	Terminate safely
09	Deep DFS branch	Depth-first behaviour
10	Greedy trap	Heuristic limitations
11	Misleading geometry	Safe A* estimate
12	Dead ends	Explore alternatives
13	Larger grid	Repeated paths
14	Decimal values	Numeric handling
15	Equal-cost routes	Deterministic ties

Independent checks

The benchmark computes reference minimum costs using Bellman-Ford relaxation, and reference minimum moves by relaxing each edge with unit cost. Every returned route is checked for its start, destination, permitted directed edges, lack of repeated states, accumulated cost and move count. AS and CUS1 must match reference minimum cost; BFS and CUS2 must match reference minimum moves.

All ninety checks passed. Eighty-four runs returned routes and six correctly reported the unreachable case. Additional regression tests cover zero-cost cycles, coincident coordinates, a starting goal, duplicate-node counting and delaying the goal test until removal. Passing this finite set is evidence of tested behaviour, not proof that every malformed input is handled.

6 Results and interpretation

Measurements were collected on Windows 11 with Python 3.13.14. Each method and case received one warm-up followed by thirty timed runs. The stored value is the median duration in milliseconds, excluding file parsing but including work performed inside the algorithm, such as heuristic setup. Measurements and routes are retained in the JSON evidence [2].

Table 2 Results on the supplied example

Method	Route	Cost	Moves	Nodes
DFS	2 > 1 > 3 > 5	15	3	9
BFS	2 > 1 > 4	10	2	5
GBFS	2 > 3 > 5	10	2	7
AS	2 > 3 > 5	10	2	9
CUS1	2 > 1 > 4	10	2	15
CUS2	2 > 3 > 5	10	2	9

DFS takes a longer and more expensive example route. The five other methods achieve cost 10 and two moves, but do different amounts of work. GBFS creates seven nodes, AS nine and CUS1 fifteen. These counts include rejected children and are not counts of unique map locations.

Table 3 Measured results on the grid case

Method	Cost	Moves	Nodes	Median ms
DFS	78	24	79	0.0556
BFS	40	8	76	0.0270
GBFS	25	8	29	0.0372
AS	17	8	299	0.4168
CUS1	17	8	736	0.5617
CUS2	25	8	573	1.1551

On this grid, AS and CUS1 both find cost 17. AS creates 299 nodes compared with 736 for CUS1, approximately 59 percent fewer. This is evidence that geometric guidance reduces created search records on this particular case. It does not establish a fixed percentage improvement across other graphs.

BFS and CUS2 both find an eight-move route, but their costs differ. BFS suppresses repeated states globally, whereas CUS2 permits alternative simple paths. Their node counts therefore reflect both their priorities and their duplicate-handling policies. Memory was not measured, and small timing differences should not be treated as stable rankings.

7 Research and limitations

Cost and move objectives

Case 06 makes the difference between objectives explicit. BFS and CUS2 choose the two-move route 1 -> 5 -> 4, costing 40. AS and CUS1 choose 1 -> 2 -> 3 -> 4, costing 3 in three moves. Neither outcome is an error when judged against the method's intended objective. Reporting only the route cost would unfairly assess the minimum-move methods.

Heuristic safety investigation

Case 11 provides evidence for scaling the AS heuristic. Its coordinates imply a large direct distance although a two-edge route costs only 2. Using raw distance as a cost estimate can overestimate. The implemented scale protects the lower-bound condition, and AS returns the minimum cost 2. This is a completed implementation improvement with a targeted regression check.

In the greedy-trap case, GBFS takes 1 -> 2 -> 5 with cost 34, while the alternative 1 -> 3 -> 4 -> 5 costs 18. The result demonstrates why looking close to a destination is insufficient to guarantee a cheap route. The grid results in Table 3 provide a separate observation about how a safe heuristic can reduce search effort.

Research status and further work

The current work includes reproducible repeated timing, independent result checks and the heuristic-scaling improvement. Whether these activities satisfy the assessed research component should be confirmed with the tutor. No visit-all-destinations solver has been implemented, and no research credit is assumed.

A possible extension is to search over a state consisting of the current location and the subset of destinations already visited. In that formulation, revisiting a location may be necessary after visiting a different subset, so location-only cycle checks would be insufficient. A complete implementation would need new goal conditions and validation against small exhaustive examples. This extension is proposed, not completed.

Known limits and missing work

No search failure was observed on the fifteen fixtures. However, this is not a claim of exhaustive correctness. The parser does not comprehensively validate every malformed numeric token or extreme numeric value. Path-based searches may create many alternative routes on dense graphs. The GUI is unverified, and peak memory has not been measured. The priority-tie interpretation and choice of custom methods should be checked against tutor guidance.

The tests are small and selected rather than representative of all navigation tasks. Conclusions about scalability require larger graphs, several graph families, and controlled comparisons with the same duplicate-handling policy. Team contribution details, signatures and required tutor progress records also remain to be completed by the group.

8 Conclusion and acknowledgements

Conclusion

The implemented program solves directed route-finding tasks using six strategies and the required command-line format. Independent checks across fifteen cases confirm the tested reachability outcomes, valid routes, minimum-cost results for AS and CUS1, and minimum-move results for BFS and CUS2. The most useful conclusion is that route quality must be judged against the chosen objective.

For minimum travel cost, AS is a reasonable default when its scaled geometric estimate provides useful guidance. CUS1 provides a minimum-cost baseline without geometry. For fewest moves, BFS is straightforward and avoids repeated states globally; CUS2 provides an informed alternative but currently explores more alternative paths. DFS and GBFS can return valid routes without guaranteeing the desired optimum.

Future performance work should compare methods under aligned duplicate-handling policies and measure frontier size or peak memory. The current data support case-specific conclusions, including the grid reduction in AS created nodes and the separation between moves and cost. They do not justify declaring one method universally fastest.

Acknowledgements and use of AI

OpenAI ChatGPT and Codex assisted with interpreting the assignment, explaining search methods, implementing DFS, GBFS, AS, CUS1 and CUS2, correcting BFS counting, creating tests, generating benchmark evidence and drafting this report. The group must review the code and report, understand the algorithms, verify the evidence, and record each member's actual contribution. AI assistance must not be described as independent student authorship.

The existing repository supplied the parser, graph model, node representation, priority frontier, BFS starting implementation, CLI, GUI file and fifteen fixture files. The Astrobyte sample report was consulted for general organisation, not as a source of this project's measured results. The AIMA algorithm reference was consulted when explaining standard frontier strategies [3].

References

[1] COS30019 teaching team. 2026. Assignment 2 Part A Tree Based Search. Supplied brief COS30019-2026 S2-A2_A (1).pdf. Used for input and output requirements, deliverables and report limits.

[2] Project implementation and measured evidence. Assignment-2-COS30019 repository, local working version, 19 October 2026. results/benchmark.json, results/comparison.md, benchmark_report.py and tests/. Repository origin: https://github.com/AntOnKnee05/Assignment-2-COS30019. Local changes are not yet pushed; the evidence files identify the results used in this report.

[3] Russell, S. and Norvig, P. Artificial Intelligence A Modern Approach. Online algorithm reference, Chapter 3, Figures 3.7, 3.9 and 3.12. https://aima.cs.berkeley.edu/algorithms.pdf. Accessed 19 October 2026. Used for standard frontier and search descriptions.